



## *Advanced Certification Programme in Agentic and Generative AI*

### **Capstone Projects List**

---

## **Implementation Guidelines for All Projects**

### **Phase 1: Foundation**

- Define the project scope, user personas, business objective, architecture block diagram, and success metrics.
- Set up the development environment using Google Colab, local Python, VS Code, or GitHub Codespaces.
- Prepare datasets, clean raw data, validate schema, and create synthetic samples where needed.
- Set up knowledge bases using vector databases such as ChromaDB or FAISS.
- Configure API keys securely using environment variables or .env files.

### **Phase 2: Core Build**

- Develop data ingestion pipelines for CSV, PDF, JSON, APIs, and database sources.
- Build RAG pipelines using LangChain or LlamaIndex with ChromaDB/FAISS retrievers.
- Design agentic workflows using LangGraph, CrewAI, AutoGen, or MCP-based tool integrations.
- Implement structured outputs using Pydantic models or JSON parsers.
- Build backend APIs using FastAPI for model inference, data retrieval, and workflow execution.

### **Phase 3: Integration and Automation**

- Connect external tools and services using APIs, webhooks, or MCP servers.
- Integrate dashboards or interfaces using Streamlit, Gradio, or React-based frontends.
- Add workflow automation using FastAPI endpoints, scheduled jobs, or event-driven triggers.
- Store processed outputs in SQLite, PostgreSQL, Airtable, or Pandas DataFrames.
- Add logging, error handling, retries, and fallback responses for robust execution.

### **Phase 4: Testing and Refinement**

- Unit-test individual pipeline components, tools, and agent nodes.
- Evaluate RAG output quality using RAGAS, retrieval accuracy, faithfulness, and answer relevance.

- Test prompts with multiple input scenarios and refine using few-shot examples.
- Validate structured outputs, dashboard updates, and API responses.
- Conduct error analysis and improve prompts, retrieval logic, thresholds, and fallback handling.

## Phase 5: Deployment and Documentation

- Deploy the application using Streamlit Cloud, HuggingFace Spaces, Docker, or cloud platforms.
- Containerize the solution using Docker for reproducibility and easier deployment.
- Document the architecture, data flow, agent workflow, APIs, and database schema.
- Prepare an evaluation report with metrics, screenshots, sample outputs, and limitations.
- Record a 5-minute walkthrough demo showing the full end-to-end workflow.

---

## Common Tools Stack

- **Programming & Development:** Python, Google Colab, VS Code, GitHub Codespaces
- **Orchestration & Agents:** LangChain, LangGraph, CrewAI, AutoGen, MCP
- **LLMs:** Groq models, OpenAI models, Llama 3, Mistral, DeepSeek, Gemma, Phi, HuggingFace Transformers
- **Embeddings:** Sentence Transformers, all-MiniLM, all-mpnet, BGE embeddings, domain-specific embeddings, OpenAI embeddings
- **Vector Databases:** ChromaDB, FAISS, Qdrant, Pinecone, Supabase
- **RAG & Evaluation:** LangChain LCEL, RAGAS, BERTScore, METEOR, scikit-learn metrics
- **Backend APIs:** FastAPI, Flask, Uvicorn, Pydantic
- **UI / Interface:** Streamlit, Gradio, React, HuggingFace Spaces
- **Databases:** SQLite, PostgreSQL, Airtable, Pandas DataFrames
- **Deployment:** Docker, HuggingFace Spaces, Streamlit Cloud, GitHub Codespaces
- **Monitoring & Logging:** Python logging, LangSmith, Langfuse, OpenTelemetry
- **Data Processing:** Pandas, NumPy, PyMuPDF, BeautifulSoup, Requests, SQLAlchemy



## Capstone Project 7

**Title:** Enhancing Customer Service with Agent-powered Chatbots

### Overview and Problem Statement:

This project aims to develop an intelligent multi-agent customer service chatbot system for e-commerce platforms using Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and agentic AI workflows. The system will provide fast, personalized, and context-aware customer support across multiple customer touchpoints such as chat interfaces, customer support portals, and web applications.

E-commerce platforms often struggle to handle large volumes of customer queries efficiently. Traditional chatbots frequently fail to provide accurate, contextual, and personalized responses, resulting in poor customer satisfaction, delayed issue resolution, and increased support costs. Businesses also face challenges in managing complex customer scenarios such as order tracking, refunds, returns, product recommendations, and escalation handling.

The proposed system will leverage multi-agent architecture, intelligent reasoning strategies, and dynamic knowledge retrieval to automate customer support workflows and improve overall customer experience.

### Dataset:

The project will utilize a combination of publicly available and synthetic datasets:

1. Customer Service Conversations:
  - Source: Kaggle's [Customer Support on Twitter](#) dataset
  - Content: Real customer service interactions from various companies
2. E-commerce Product Catalog:
  - Source: [Amazon product dataset](#) (a subset focused on electronics and home appliances)
  - Content: Product descriptions, specifications, prices, and customer reviews
3. Synthetic E-commerce Queries:
  - Source: Generated using GPT-5 or similar language models
  - Content: Simulated customer queries specific to e-commerce scenarios

### Methodology:

1. Data Preparation:

- a. Collect and preprocess customer support conversations and product catalog datasets
  - b. Clean and organize product and customer interaction data
  - c. Generate synthetic customer queries for training and testing scenarios
2. Agentic Architecture Development:
  - a. Design multi-agent workflows for customer query handling
  - b. Create specialized agents for order tracking, returns, escalation handling, and product recommendations
  - c. Implement task decomposition and reasoning strategies
3. RAG Pipeline Integration:
  - a. Build vector database for product and support knowledge retrieval
  - b. Retrieve relevant contextual information during customer interactions
  - c. Integrate semantic search for accurate response generation
4. AI Chatbot Development:
  - a. Develop conversational chatbot using LLMs
  - b. Maintain customer conversation history and context awareness
  - c. Generate personalized and context-aware responses
5. Integration and Automation:
  - a. Build web-based chatbot interface using Streamlit or Gradio
  - b. Integrate APIs for order lookup and customer support workflows
  - c. Automate support workflows and escalation pipelines
6. Evaluation:
  - a. Measure chatbot performance using response accuracy, latency, and customer satisfaction metrics
  - b. Evaluate agent coordination and recommendation quality

### Challenges:

1. Task Decomposition: Break complex customer queries into manageable subtasks for specialized agents
2. Multi-Agent Coordination: Ensure efficient collaboration between multiple AI agents
3. Context Retention: Maintain customer conversation history and contextual understanding during interactions
4. Product Knowledge Retrieval: Efficiently manage and retrieve product information using RAG pipelines
5. Scalability: Support high-volume customer interactions with low response latency
6. Personalization: Generate personalized recommendations and responses based on customer history
7. Hallucination and Accuracy: Prevent inaccurate or misleading AI-generated responses



## References:

1. Lewis, Patrick, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv preprint arXiv:2005.11401.
2. Henderson, Matthew, et al. 2019. "A repository of conversational datasets." arXiv preprint arXiv:1904.06472.
3. Serban, Iulian Vlad, et al. 2017. "A deep reinforcement learning chatbot." arXiv preprint arXiv:1709.02349.